

Numerical Linear Algebra II: 01 Floating Point Arithmetic and Algorithm Stability

Jakub Kruzik
jakub.kruzik@vsb.cz

2026
Ostrava

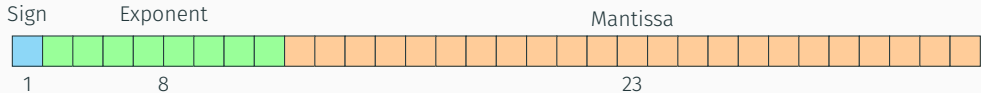
Representation of Numbers on Computers - Binary Floating Point (IEEE 754)

Representation:

$$(-1)^{\text{sign}} \times (1.\text{mantissa}) \times 2^{\text{exponent}-\text{bias}}$$

Single precision (32-bit):

- Sign: 1 bit
- Exponent: 8 bits (bias = 127)
- Mantissa: 23 bits



Special values:

- Exponent all 0s $\Rightarrow$ denormals / 0
- Exponent all 1s $\Rightarrow \infty$ / NaN

Example: Convert 5.75 to Single Precision (IEEE 754)

Step 1: Convert to binary

$$5.75_{10} = 101.11_2$$

Step 2: Normalize to scientific notation (base 2)

$$101.11_2 = 1.0111_2 \times 2^2$$

Step 3: Determine sign, exponent, and mantissa

- Sign = 0 (positive number)
- Exponent = $2 + 127 = 129 = 10000001_2$
- Mantissa = digits after the leading 1: 011100000000000000000000

Step 4: Combine into 32-bit IEEE 754 single precision

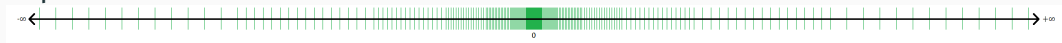
0 10000001 011100000000000000000000

sign exponent mantissa

FP Formats Properties

Format	Precision	Digits (bin)	Digits (dec)	Exp. range	Max value	Min normal
binary16	Half	11	3.31	$[-14, +15]$	6.55×10^4	6.10×10^{-5}
binary32	Single	24	7.22	$[-126, +127]$	3.40×10^{38}	1.18×10^{-38}
binary64	Double	53	15.95	$[-1022, +1023]$	1.80×10^{308}	2.23×10^{-308}
binary128	Quadruple	113	34.02	$[-16382, +16383]$	1.19×10^{4932}	3.36×10^{-4932}

Representable numbers:



Beware Rounding:

$$0.1 + 0.2 \neq 0.3$$

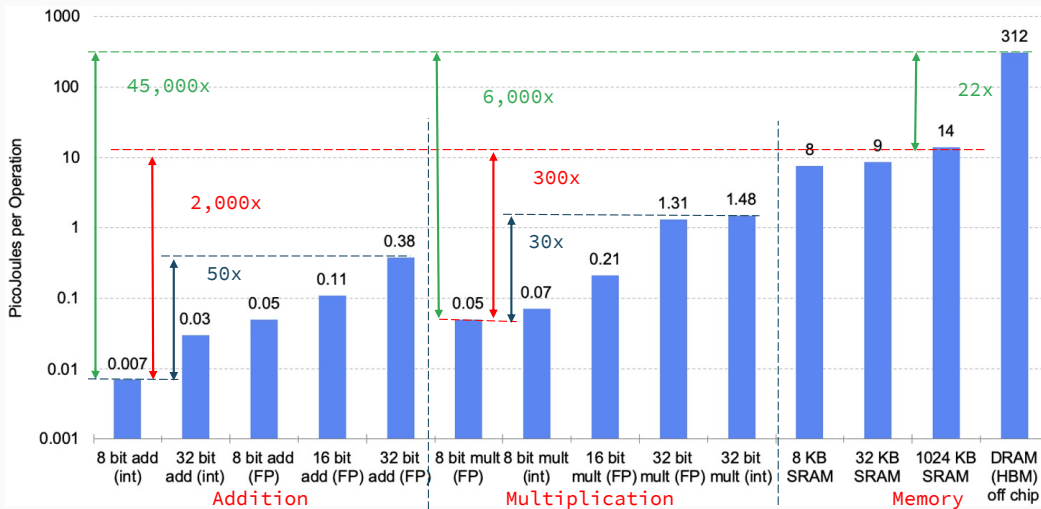
Beware Associativity:

$$\begin{aligned}(10^{16} - 10^{16}) + 1.0 &= 1.0 & 10^{16} + (-10^{16} + 1.0) &= 0.0 \\ (10^{300} \times 10^{300}) \times 10^{-300} &= +\infty & 10^{300} \times (10^{300} \times 10^{-300}) &= 10^{300}\end{aligned}$$

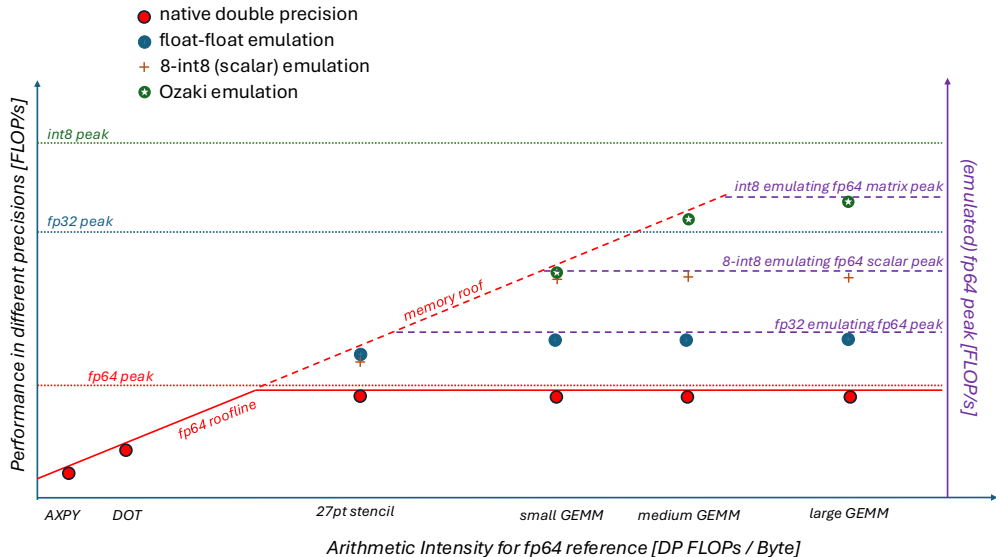
Mixed Precision Computing Motivation

Metric	Ampere (2020)	Hopper (2022)	Blackwell (2024)	Rubin (2026*)
FP16 Matrix	312	1000	2500	4000
FP32 Vector	20	67	80	130
FP64 Vector	10	34	40	33

Mixed Precision Computing Motivation



Emulation of Higher Precision



Mixed Precision Formats

	754	BF16	TF32	OCP	P3109	AGQ	HiF8	TSL	LNSM	Posit	Takum
Hardware	✓	✓	✓	✓	✗	✓	●	✓	●	✓	✗
Not Patented	✓	✓	✓	✓	✓	?	✓	✓	✗	✓	✓
Portable	✗	✗	✗	✗	✓	?	✓	✓	✓	✓	✓
Well Defined	✓	✗	✗	✗	✓	✗	✗	✗	✗	✓	✓
Parameterized	✗	✗	✗	✗	✓	?	✗	✗	✓	✓	✓
Uniqueness	✗	✗	✗	✗	✓	?	✓	✗	✓	✓	✓
Logarithmic format	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Saturation Modes Specified	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	✓
Rounding Modes Specified	B	✗	✗	C	A	?	✗	A	✓	●	●
Block formats	✗	✗	✗	●	✓	✗	✗	✗	✗	✗	✗
Format Convertible	✓	✓	✓	✗	✓	?	✓	✓	✗	✓	✓
Precision Convertible	✓	●	●	✗	✓	?	●	✗	●	✓	✓
High precision accumulator	-	-	✓	-	●	-	-	-	-	✓	✓
Numerical Analysis	✓	✓	✓	✓	✓	✓	✗	✓	✗	●	●
Composable for emulation	✓	✓	✓	✓	✓	✓	✗	✓	✗	✗	●
$a = b \Leftrightarrow a - b = 0$	✓	●	●	✓	✓	?	✗	?	✓	✓	✓
$e_{\max} + e_{\min}$	1	1	1	{1, 2}	0	?	0	?	?	0	0

Forward vs Backward Error (General)

Given $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$

$$\mathbf{y} = f(\mathbf{x})$$

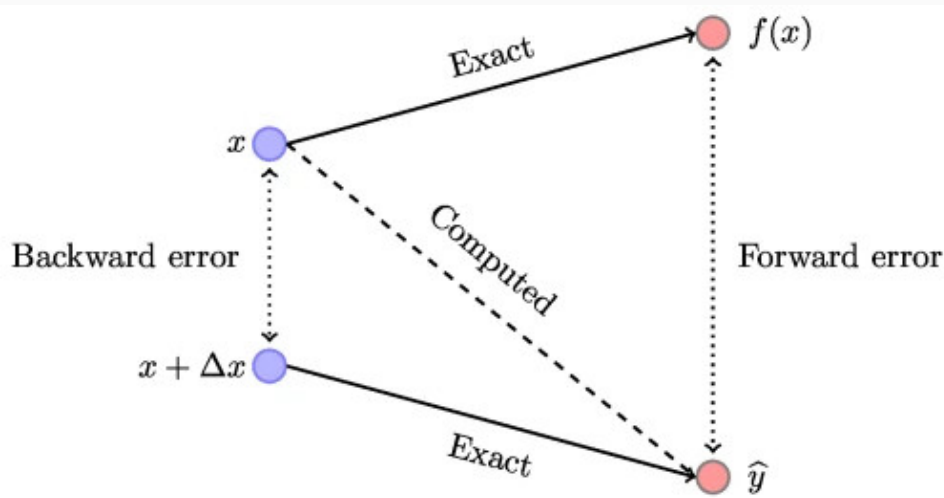
- **Forward Error:** Difference between computed solution $\hat{\mathbf{y}}$ and true solution $\mathbf{y}$:

$$\epsilon_{\text{forward}} = \|\hat{\mathbf{y}} - \mathbf{y}\|$$

- **Backward Error:** Smallest perturbation $\Delta \mathbf{x}$ that makes $\hat{\mathbf{y}}$ exact:

$$\epsilon_b(\hat{\mathbf{y}}) = \min\{\epsilon : \hat{\mathbf{y}} = f(\mathbf{x} + \Delta \mathbf{x}), \|\Delta \mathbf{x}\| \leq \epsilon \|\mathbf{x}\|\}$$

Forward vs Backward Error (General)



source: <https://nhigam.com/2020/03/25/what-is-backward-error/>

Forward vs Backward Error for Linear Systems

Given

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \text{ where } \mathbf{A} \in \mathbb{R}^{n \times n} \text{ and } \mathbf{b} \in \mathbb{R}^n.$$

- **Forward Error:** Difference between computed solution $\hat{\mathbf{x}}$ and true solution $\mathbf{x}$:

$$\epsilon_{\text{forward}} = \|\Delta\mathbf{x}\| = \|\hat{\mathbf{x}} - \mathbf{x}\|$$

- **Residual:** How far $\hat{\mathbf{x}}$ fails to satisfy the original system:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$$

- **Relative Backward Error:** Smallest perturbation $\Delta\mathbf{A}, \Delta\mathbf{b}$ that makes $\hat{\mathbf{x}}$ exact:

$$\epsilon_b(\hat{\mathbf{x}}) = \min\{\epsilon : (\mathbf{A} + \Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta\mathbf{b}, \|\Delta\mathbf{A}\| \leq \epsilon\|\mathbf{A}\|, \|\Delta\mathbf{b}\| \leq \epsilon\|\mathbf{b}\|\}$$

It can be shown:

$$\epsilon_b(\hat{\mathbf{x}}) = \frac{\|\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}\|}{\|\mathbf{b}\| + \|\mathbf{A}\|\|\hat{\mathbf{x}}\|} = \frac{\|\Delta\mathbf{A}_{\min}\|}{\|\mathbf{A}\|} = \frac{\|\Delta\mathbf{b}_{\min}\|}{\|\mathbf{b}\|}$$

Why Backward Error?

1. Computable without knowing the true solution

- Forward error needs $\mathbf{x}$ (the exact solution), which we don't know.
- Backward error only needs $\mathbf{A}$, $\mathbf{b}$, and $\hat{\mathbf{x}}$ (the computed solution).

2. Natural measure of algorithm stability

- An algorithm is *backward stable* if it solves a nearby problem exactly:

$$(\mathbf{A} + \Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta\mathbf{b}, \quad \|\Delta\mathbf{A}\|, \|\Delta\mathbf{b}\| \text{ small.}$$

- This matches how floating-point arithmetic perturbs data.

3. Separates problem difficulty from algorithm quality (next slide)

- Forward error $\leq \kappa(\mathbf{A}) \cdot$ (backward error),
- $\kappa(\mathbf{A})$ = sensitivity of the problem,
- backward error = performance of the algorithm.

Error Analysis - Perturbation of only $\mathbf{b}$

$\hat{\mathbf{x}}$ is the solution of the perturbed system

$$\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b}$$

Subtracting $\mathbf{Ax}(=\mathbf{b})$ from both sides, we get

$$\mathbf{A}\Delta\mathbf{x} = \Delta\mathbf{b} \quad \Rightarrow \quad \Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b}$$

Take norms:

$$\|\Delta\mathbf{x}\| = \|\mathbf{A}^{-1}\Delta\mathbf{b}\| \leq \|\mathbf{A}^{-1}\| \|\Delta\mathbf{b}\| \quad (1)$$

Use $\mathbf{Ax} = \mathbf{b}$ again, to estimate

$$\|\mathbf{b}\| = \|\mathbf{Ax}\| \leq \|\mathbf{A}\| \|\mathbf{x}\| \Rightarrow \frac{1}{\|\mathbf{x}\|} \leq \|\mathbf{A}\| \frac{1}{\|\mathbf{b}\|} \quad (2)$$

Combining (1) and (2) gives the estimate of the relative forward error

$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \|\mathbf{A}\| \|\mathbf{A}^{-1}\| \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|}$$

Error Analysis - Perturbation of only b

In this case, the unique perturbation Δb

$$A\hat{x} = b + \Delta b \Rightarrow \Delta b = A\hat{x} - b = -\hat{r},$$

where $\hat{r}$ is the residual.

Then

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\Delta b\|}{\|b\|} = \|A\| \|A^{-1}\| \frac{\|\hat{r}\|}{\|b\|}$$

$\kappa(A) = \|A\| \|A^{-1}\|$ is the condition number.

Recall that $\frac{\|\Delta b_{\min}\|}{\|b\|}$ is, in this case, the relative backward error.

$$\mathbf{A}\hat{\mathbf{x}} = (\mathbf{A} + \Delta\mathbf{A})(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b}$$

If $\|\mathbf{A}^{-1}\|_p \|\Delta\mathbf{A}\|_p < 1$ then¹

$$\frac{\|\Delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \frac{\kappa_p(\mathbf{A})}{1 - \kappa_p(\mathbf{A}) \frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \left(\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|} \right)$$

If $\kappa_p(\mathbf{A})$ is small, we say that the linear system is **well conditioned**.

Otherwise, we say that the linear system is **ill conditioned**.

If an algorithm is backward stable, it can be shown that

$$\frac{\|\Delta\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \leq \kappa_p(\mathbf{A}) \epsilon_M,$$

where ϵ_M is the unit roundoff ($2^{-53} \approx 1.11\text{e-}16$ for double).

¹with any p -norm, i.e., $1 \leq p \leq \infty$. The same can be done on the previous slides.

Key Takeaways

- Forward error measures how far the computed solution is from the true solution.
- Residual measures how well the computed solution satisfies the original system.
- Backward error measures the minimal perturbation in the data making the computed solution exact.
- For linear systems with only $\mathbf{b}$ perturbations, relative backward error = norm of relative (to $\mathbf{b}$!) residual
- Backward error is a natural measure of algorithm stability.